AI Assisted Coding

Assignment-12.3

Name: A. sai shrehan

Rollno:2303A52299

Batch:45

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).

2. Implement the following sorting algorithms using AI assistance:

o Quick Sort

o Merge Sort

3. Measure and compare runtime performance for large datasets.

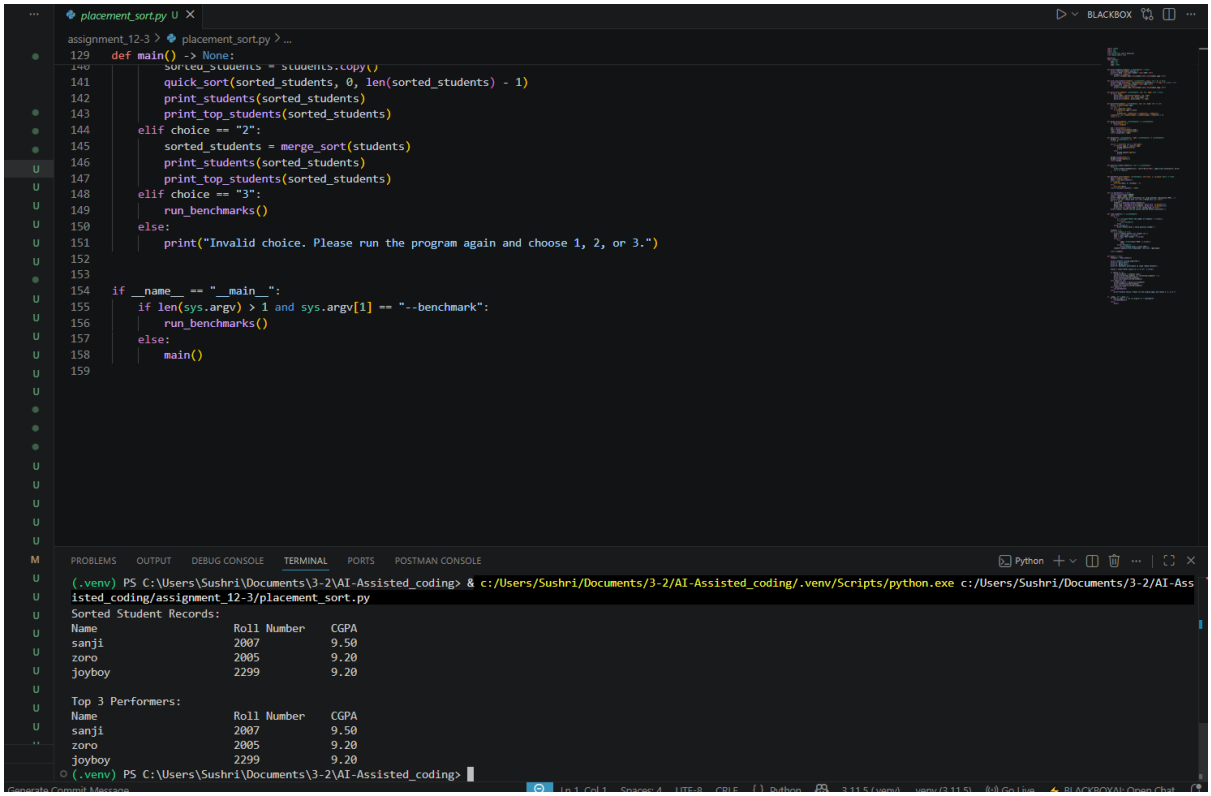
4. Write a function to display the top 10 students based on CGPA.

Expected Outcome

• Correctly sorted student records.

• Performance comparison between Quick Sort and Merge Sort.

• Clear output of top-performing students.



Task 2: Implementing Bubble Sort with AI Comments

• Task: Write a Python implementation of Bubble Sort.

• Instructions:

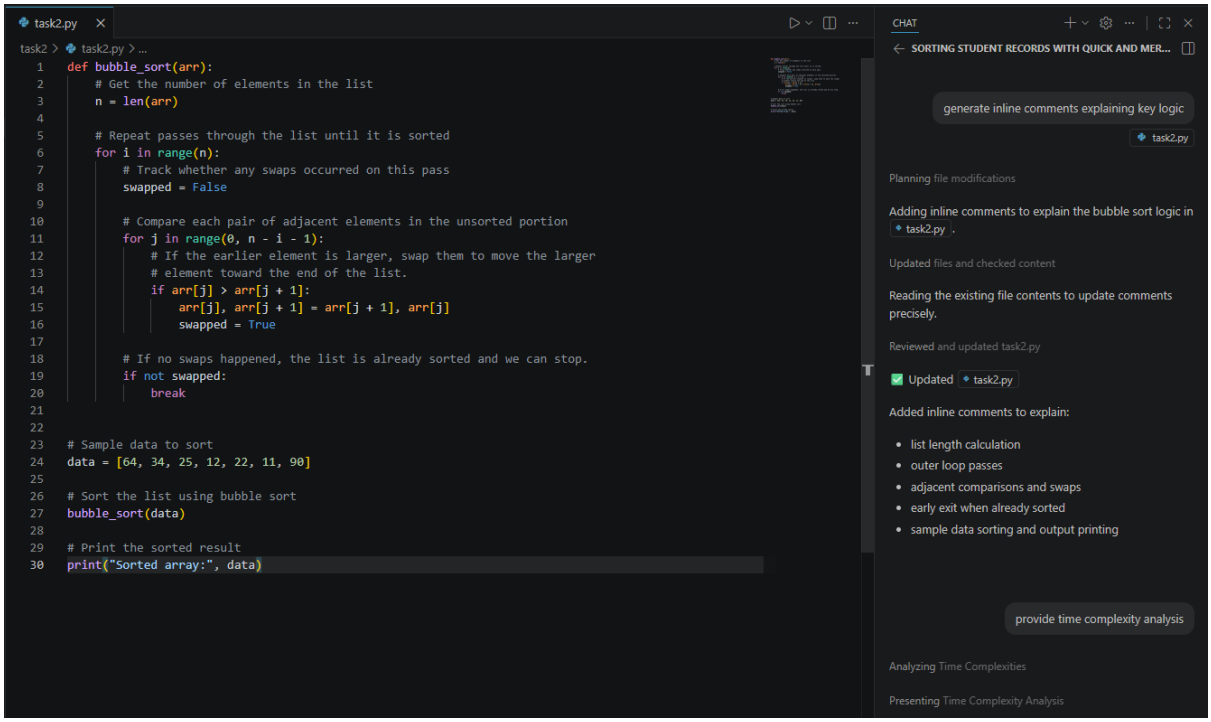
• Students implement Bubble Sort normally.

• Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).

• Request AI to provide time complexity analysis.

• Expected Output:

• A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.



Task 3: Quick Sort and Merge Sort Comparison

• Task: Implement Quick Sort and Merge Sort using recursion.

• Instructions:

• Provide AI with partially completed functions for recursion.

• Ask AI to complete the missing logic and add docstrings.

• Compare both algorithms on random, sorted, and reverse-sorted lists.

• Expected Output:

• Working Quick Sort and Merge Sort implementations.

• AI-generated explanation of average, best, and worst-case complexities.

task3.py

task3 > task3.py > ...

```python
def quicksort(arr):

    Returns:
        A new sorted list containing the same values as ``arr``.
    """
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quicksort(left) + middle + quicksort(right)


def merge_sort(arr):
    """Sort a list using the merge sort algorithm.

    The function returns a new list in ascending order. It splits the input
    list recursively and then merges the sorted halves.

    Args:
        arr: A list of comparable values.

    Returns:
        A new sorted list containing the same values as ``arr``.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)


def merge(left, right):
    """Merge two sorted lists into a single sorted list.

    Args:
        left: A sorted list.
        right: A sorted list.

    Returns:
        A new sorted list containing all values from ``left`` and ``right``
```

CHAT

SORTING STUDENT RECORDS WITH QUICK AND MER...

memory.

For your bubble sort file, the key point is that the optimization with `swapped = False` makes the best-case linear instead of always quadratic.

complete the missing logic and add docstrings.

task3.py

Modifying task3.py

Inspecting task3 script and then adding missing logic plus docstrings for clarity.

Reviewed task3.py and determined missing logic

Updating `task3.py` with complete runtime logic, docstrings, and a sample execution block.

Edited `task3.py` +50 -4

Updated `task3.py`

What was added:

- docstrings for `quicksort`, `merge_sort`, and `merge`
- a complete sample execution block under `if __name__ == "__main__":`
- clear demonstration of sorting with both algorithms

This makes the file self-contained and easier to understand.

task3.py

Describe what to build

Agent Auto

Local Default Approvals

assignment_12-3 > task3.py > quicksort

```python
def quicksort(arr):
    The function returns a new list in ascending order.
    It uses the middle element as the pivot and partitions the list into
    elements less than, equal to, and greater than the pivot.

    Args:
        arr: A list of comparable values.

    Returns:
        A new sorted list containing the same values as ``arr``.
    """
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quicksort(left) + middle + quicksort(right)


def merge_sort(arr):
    """Sort a list using the merge sort algorithm.

    The function returns a new list in ascending order. It splits the input
    list recursively and then merges the sorted halves.

    Args:
        arr: A list of comparable values.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

Python

```
PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/.venv/Scripts/python.exe c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/assignment_12-3/task3.py
Original list: [33, 10, 55, 71, 29, 8, 90]
Quick sort result: [8, 10, 29, 33, 55, 71, 90]
Merge sort result: [8, 10, 29, 33, 55, 71, 90]

Comparison of sorting algorithms:
Input type        Quick Sort time (s)   Merge Sort time (s)
random                     0.010150              0.013377
sorted                     0.008720              0.007309
reverse-sorted             0.004518              0.006703
PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:\Users\Sushri\Documents\3-2\AI-Assisted_coding\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding>
```

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

Task:

• Use AI to suggest the most efficient search and sort algorithms for this use case.

• Implement the recommended algorithms in Python.

• Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

• A table mapping operation → recommended algorithm → justification.

• Working Python functions for searching and sorting the inventory.

assignment_12-3 > task4.py > Inventory > search_by_id

```python
from dataclasses import dataclass
from typing import Dict, List, Optional

@dataclass
class Product:
    product_id: str
    name: str
    price: float
    stock_quantity: int


class Inventory:
    """Inventory system with fast search and stable sorting."""

    def __init__(self, products: List[Product]):
        self.products = products
        self.id_index: Dict[str, Product] = {}
        self.name_index: Dict[str, List[Product]] = {}
        self._build_indices()

    def _build_indices(self) -> None:
        """Build lookup dictionaries for fast product search."""
        for product in self.products:
            self.id_index[product.product_id] = product
            key = product.name.lower()
            self.name_index.setdefault(key, []).append(product)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE Python

```
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/.venv/Scripts/python.exe c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/assignment_12-3/task4.py
Product(product_id='P1001', name='Widget', price=9.99, stock_quantity=120)
Product(product_id='P1003', name='Widget', price=9.99, stock_quantity=50)
Product(product_id='P1002', name='Gadget', price=14.99, stock_quantity=80)
Product(product_id='P1004', name='Thingamajig', price=24.99, stock_quantity=200)

Products sorted by stock quantity descending:
Product(product_id='P1004', name='Thingamajig', price=24.99, stock_quantity=200)
Product(product_id='P1001', name='Widget', price=9.99, stock_quantity=120)
Product(product_id='P1002', name='Gadget', price=14.99, stock_quantity=80)
Product(product_id='P1003', name='Widget', price=9.99, stock_quantity=50)
Product(product_id='P1005', name='Doodad', price=4.99, stock_quantity=30)
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding>
```

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

• Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).

• Implement sorting algorithms to rank stocks by percentage change.

• Implement a search function that retrieves stock data instantly when a stock symbol is entered.

• Optimize sorting with Heap Sort and searching with Hash Maps.

• Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

assignment_12-3 > task5.py > ...

```python
import random
import string
import time
from dataclasses import dataclass
from typing import Dict, List, Optional


@dataclass
class Stock:
    symbol: str
    opening_price: float
    closing_price: float

    @property
    def percentage_change(self) -> float:
        """Compute percentage gain/loss from opening to closing price."""
        if self.opening_price == 0:
            return 0.0
        return ((self.closing_price - self.opening_price) / self.opening_price) * 100


def generate_stock_data(count: int) -> List[Stock]:
    """Generate simulated stock data for analysis."""
    symbols = set()
    stocks: List[Stock] = []
    while len(stocks) < count:
        symbol = ''.join(random.choices(string.ascii_uppercase, k=3))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/.venv/Scripts/python.exe c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/assignment_12-3/task5.py
WYZ       155.08  124.11      -19.97

Sorting performance comparison:
Algorithm               Time (s)
Heap Sort               0.054834
Built-in sorted()       0.001444

Trade-offs:
- Heap Sort: O(n log n) worst-case, no extra temporary sort data structure beyond the copy, but slower in Python due to manual loops.
- built-in sorted(): also O(n log n), highly optimized in C, and generally faster for Python objects.

Search performance comparison:
Search type             Time (s)
Hash map lookup         0.000006

Trade-offs:
```